weather in a specific city. Whether the AI model chooses to call this tool depends on the model's performance and reliability. This call results in a streaming React component based on the fetch state of the weather API.

Listing 4.14 Providing the weather info tool details

```
const supportedModel = getSupportedModel(provider, model);
const result = await streamUI({
  model:  supportedModel,
  system: `
    You are a helpful weather assistant.
You can provide weather information for cities.
    If a user asks about the weather in a specific city,
use the 'getWeather' function to fetch the data.
    Always interpret temperatures in Celsius.
  `,
  messages: [
    { role: 'user', content: 'What\'s the weather like in Paris today?' }
  ],
  tools: {.
    getWeather: {.
      description: 'Get the current weather for
a specific city',
      parameters: z.object({.
        city: z.string().describe('The name of the city'),
      }),
      generate: async function* ({ city }) {.
        yield <LoadingSpinner />

        // Simulating API call
        await sleep(1000)
        const weatherData = await fetchWeatherData(city)

        return <WeatherCard city={city} temperature={weatherData.temperature}
condition={weatherData.condition} />
      }
    }
  }
})
```

The tools object defines custom functions that the AI can call.

getWeather is the name of the tool that the AI will use to fetch weather data.

The description helps the AI understand when and how to use this tool.

parameters uses Zod schema to define and validate the expected input (a city name).

generate is an async generator function that produces the tool's output, including interim loading states.

The `tools` object defines custom functions that the AI can call. In this case, we define a single tool called `getWeather`, which accepts a `description` and `parameters` using the Zod schema. Lastly, the `generate` function is an async generator function that fetches the weather data and returns a `WeatherCard` component with the fetched data.

This tool demonstrates the AI's ability to request and present real-time weather data in a structured format, seamlessly integrating external information into the conversation flow. This also illustrates the versatility of the Vercel AI SDK's interface for AI model interactions. By utilizing this common interface, we can easily swap the `model`